Object oriented programming and conceptual modeling best practices

conceptual modeling Explains what it is like capturing ideas and concepts to understand, visualize, and document real-world systems, when it applies at any stages of software development and from multiple perspectives, and how UML groups languages such as use case diagrams, class diagrams, interactive diagrams, and state charts

Design and architecture Principles

- **Organize software into loosely coupled layers:** User interface, application logic, persistence, communications, and operating system.
- **Prefer aggregation over inheritance:** aggregation offers greater flexibility since it can be modified at runtime
- **Program to an interface or abstraction:** reduces coupling and eliminates unnecessary dependencies
- **Use readable identifiers:** classes in PascalCase, variables in camelCase, and methods using verbs
- **Executable unit tests** help find design errors, verify correctness, document usage, and enable regression testing

Introduction to the unified modeling language

Nature of UML: "It's an expresive, semi-formal object-oriented modeling language that is process-independent and customizable

Software engineering modeling perspectives

- **Analysis:** Description of the problem domain used real-world objects
- **Specification:** Logical description of the software system and automation boundaries
- **Implementation:** Description of solution-oriented software components

Types of diagrams: divided into structural (Classes, Objects, Components, Deployment) and behavioral (Use Cases, Sequence, Collaboration, States, Activities)

Best Practices: the purpose of diagrams into communicate, maintain meaningful names, avoid crossing lines, and organize elements logically

Basic Use Cases

Actors: External elements (users, hardware, other systems) that interact with the systems to achieve goals

Use cases: Capture user-visible funcionality by answering the WHAT and WHY. They should not be confused with system interactions (which answer the HOW)

Relationships in use case diagrams

- **Generalization:** Between actors or use cases (a specific use case or actor in a subset of an more general one)
- **Inclusion** a use case strictly requires the goal defined by another

<<include>>

- Extension: a use case adds optional behavior o variations to another under specific conditions

Class Diagrams

Object and Classes: Objects are foundational building blocks with identity, state, and behavior, classes are abstractions of collections of objects showing common characteristics

Class syntax: Divided into compartments that include name, attributes, operations and responsabilities

Types of relationship:

- Generalization: Inheritance or specification "isa"
- Association: Structural links between classes with names and directionality
- Aggregation: Part-whole relationship "is part of" of an asymmetric and transitive nature
- Dependency: a usage relationship where a change in one element may affect another

UML Quick Reference Guide

Functions as a compact summary table of UML, visual components for analysis and design activities

Use cases: Actors, associations, inclusion and extension dependencies

Class diagram: Classes, attributes, operations, generalizations, associations with multiplicities, aggregations, and compositions

Activity diagram: Initial nodes, action nodes, control/object flows, decision nodes, forks, join, and signal sending/receiving

Interaction diagram: Objects, communication channels, and synchronous, asynchronous, creation, or lost messages

State charts: Initial/final states, transitions (with triggers, guards, and actions) internal activities, and entry/exit transitions